



Building Java Programs
A BACK TO BASICS APPROACH 5e

Fundamentals of Computer Science

II

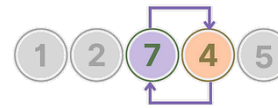
08 - Sorting

Instructor: Varik Hoang

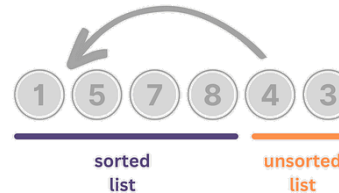
Objectives

- Search
 - Sequential Search
 - Binary Search
- Comparator
- Swapping Function
- Sorting Algorithms
 - Bubble Sort
 - Selection Sort
 - Insertion Sort
 - Quick Sort
 - Merge Sort

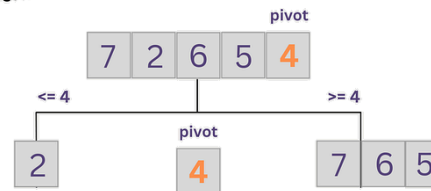
BUBBLE SORT



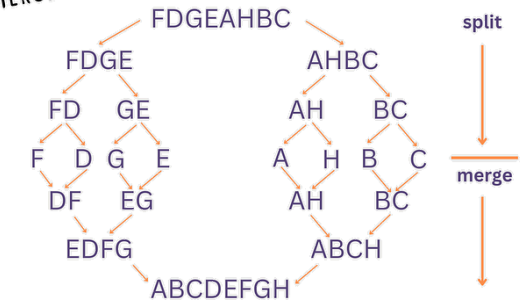
INSERTION SORT



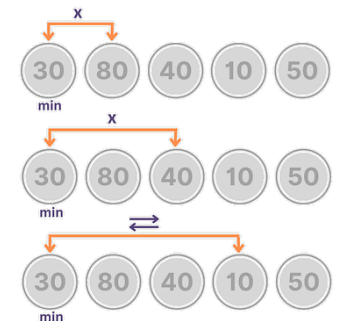
QUICKSORT



MERGE SORT



SELECTION SORT

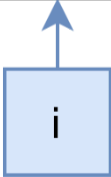




Sequential Search

- Locates a target value in a list by examining each element from start to finish.
 - How many elements will it need to examine?
 - Example: Searching the array below for the value 42
- Notice that the **array is sorted**.

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	100



Binary Search

- Locates a target value in a sorted list by eliminating half of the list from consideration.
 - How many elements will it need to examine?
 - Example: Searching the array below for the value 42
- Notice that the **array is sorted**. Could we take advantage of this?

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	-4	2	7	10	15	20	22	25	30	36	42	50	56	68	85	92	100

Diagram illustrating binary search steps on a sorted array. Blue boxes labeled 'min', 'middle', 'min', 'middle', 'max', and 'max' are positioned below the array with arrows pointing to specific indices. The first 'min' points to index 0. The first 'middle' points to index 8. The second 'min' points to index 10. The second 'middle' points to index 11. The first 'max' points to index 12. The second 'max' points to index 16.



Comparator

- Sometimes, the default ordering is not what you want.
- Example: The following code sorts the strings in a case-sensitive order, so the uppercase letters come first. We may want a case-insensitive ordering instead.

```
String[] strings = {"Foxtrot", "alpha", "echo", "golf",  
                    "bravo", "hotel", "Charlie", "DELTA"};
```

```
Arrays.sort(strings);  
System.out.println(Arrays.toString(strings));
```

- Output: [Charlie, DELTA, Foxtrot, alpha, bravo, echo, golf, hotel]



Comparator (cont.)

- The Comparator interface in java.util describes a method for comparing two objects of a given type:

```
public interface Comparator<Type>
{
    public int compare(Type o1, Type o2);
}
```

- Note that **Comparator** is a generic interface. **Type** will be replaced by the type of object you are actually comparing.
- The compare method's job is to decide the relative ordering of the two given objects and return an appropriate integer:
 - < 0 if o1 comes before o2
 - $= 0$ if o1 and o2 are equivalent
 - > 0 if o1 comes after o2

Comparator (cont.)

- The following Comparator (two versions) compares Strings, ignoring case:

```
public class CaseInsensitiveComparator
implements Comparator<String>
{
    @Override
    public int compare(String s1, String s2)
    {
        return s1.toLowerCase().compareTo(s2.toLowerCase());
    }
}
```

```
public class CaseInsensitiveComparator
implements Comparator<String>
{
    @Override
    public int compare(String s1, String s2)
    {
        return s1.compareToIgnoreCase(s2);
    }
}
```

Comparator

- The sorting algorithm will use that comparator to order the elements

```
String[] strings = {"Foxtrot", "alpha", "echo", "golf",  
                    "bravo", "hotel", "Charlie", "DELTA"};  
  
Arrays.sort(strings);  
System.out.println(Arrays.toString(strings));  
Arrays.sort(strings, new CaseInsensitiveComparator());  
System.out.println(Arrays.toString(strings));
```

- Output:
 - [Charlie, DELTA, Foxtrot, alpha, bravo, echo, golf, hotel]
 - [alpha, bravo, Charlie, DELTA, echo, Foxtrot, golf, hotel]

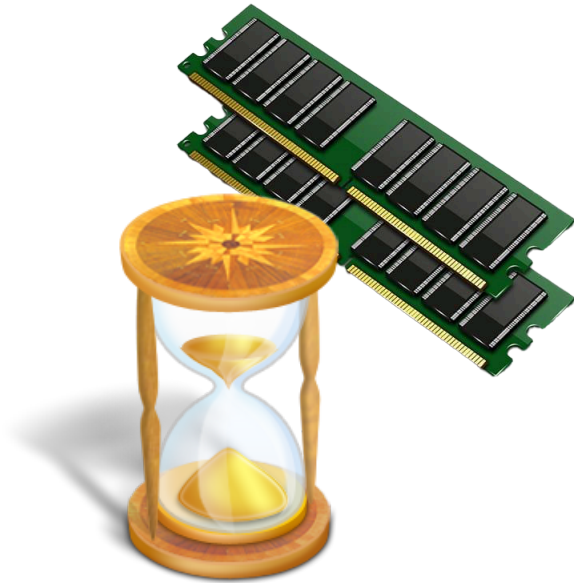
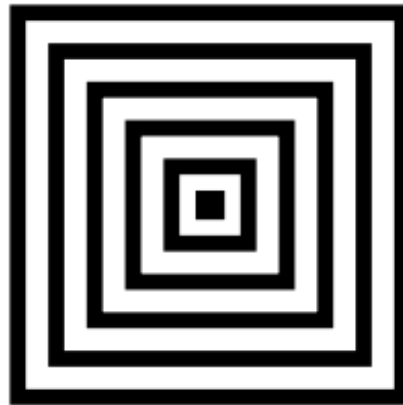
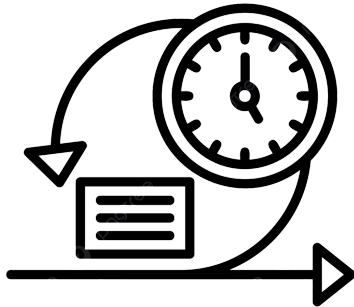


Comparator - Exercise

- 1 How would you force the sort to descending order?
 - Create a class name **DescendingComparator** to implement the comparator
 - The expected output would be:
[**hotel**, **golf**, **Foxtrot**, **echo**, **DELTA**, **Charlie**, **bravo**, **alpha**]
- 2 How would you force the sort to reverse the string and sort?
 - Create a class name **StringReverseComparator** to implement the comparator
 - The expected output would be:
[**DELTA**, **alpha**, **Charlie**, **golf**, **hotel**, **echo****o**, **bravo****o**, **Foxtrot**]

Sorting Algorithms

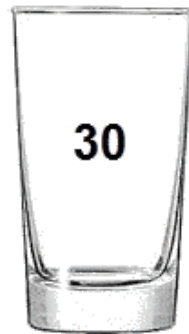
- Memory Usage (RAM)
- Complexity (Time & Space)
- Approach (Iteration & Recursion)



Swapping Function

```
private static void swap(int[] array, int a, int b)
{
    int temp = array[a];
    array[a] = array[b];
    array[b] = temp;
}
```

First Step



A



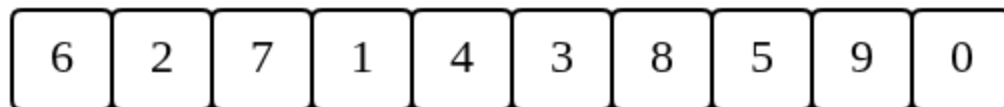
B



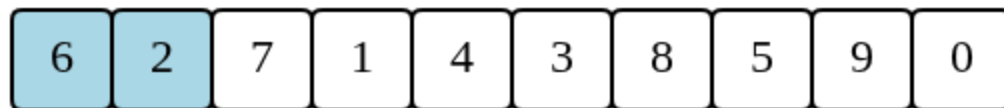
Temp



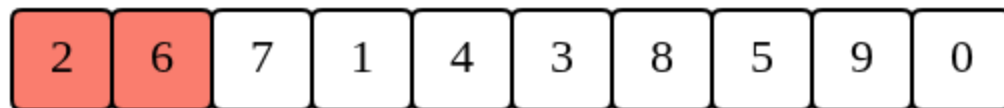
Bubble Sort - Example



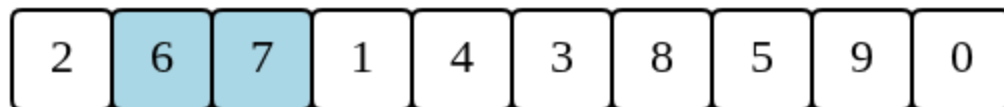
provide the unsorted array



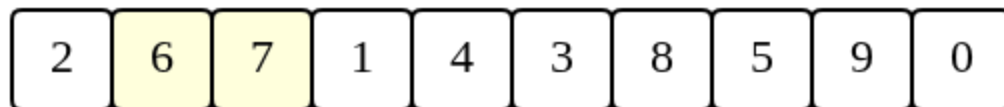
compare first pair of two elements



swap them because 2 should go before 6



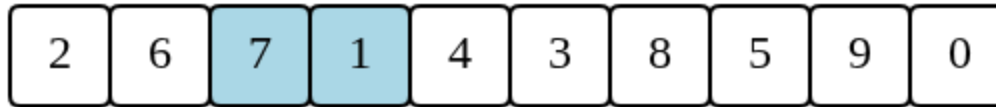
compare next pair of elements



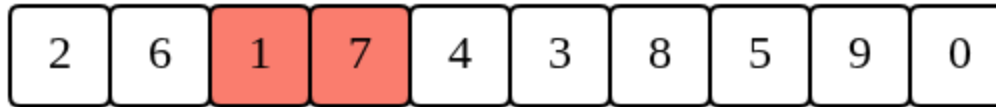
do nothing since 6 and 7 are in right order



Bubble Sort - Example (cont.)



compare next pair of elements



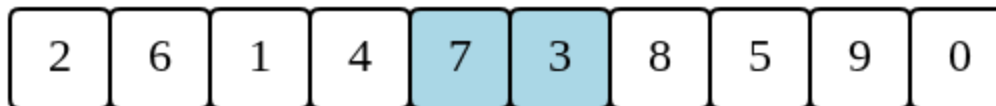
swap them because 1 should go before 7



compare next pair of elements

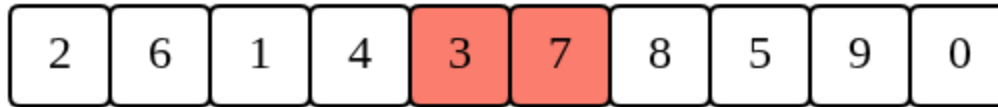


swap them because 4 should go before 7

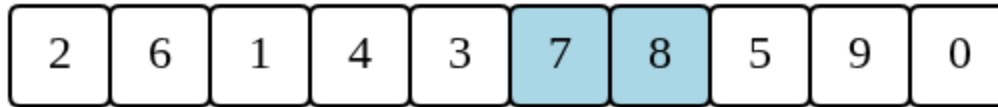


compare next pair of elements

Bubble Sort - Example (cont.)



swap them because 3 should go before 7



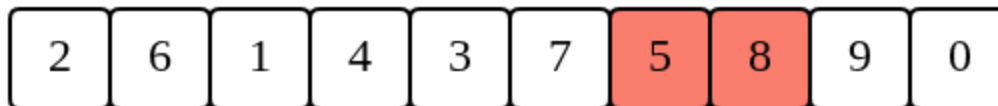
compare next pair of elements



do nothing since 7 and 8 are in right order

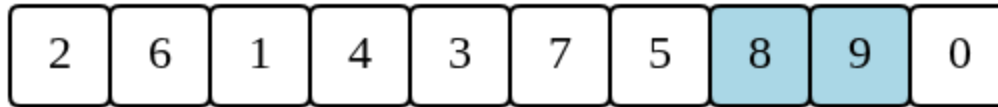


compare next pair of elements

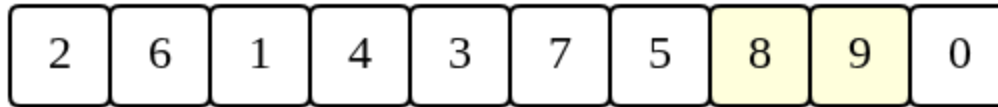


swap them because 5 should go before 8

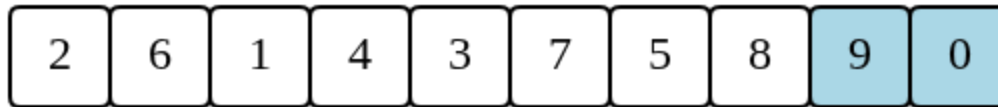
Bubble Sort - Example (cont.)



compare next pair of elements



do nothing since 8 and 9 are in right order



compare next pair of elements



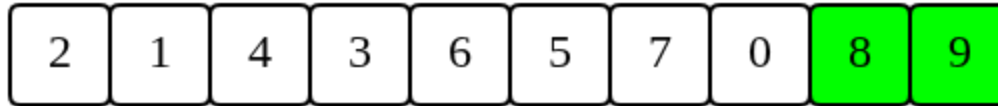
swap them because 0 should go before 9



the array after 1st iteration



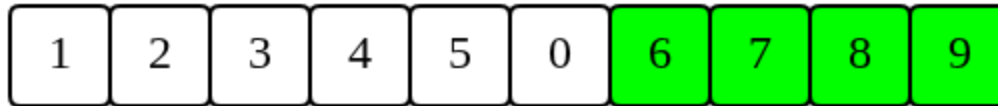
Bubble Sort - Example (cont.)



the array after 2nd iteration



the array after 3rd iteration



the array after 4th iteration



the array after 9th iteration



the array is now sorted



Bubble Sort - Observation

- The main idea of the algorithm is to push the maximum value to the end of the array for each iteration
- The original implementation was to repeat ($n - 1$) iterations which n is the number of elements in the array
- The optimized implementation has a flag to finish the sort when the iteration does not make any swapping

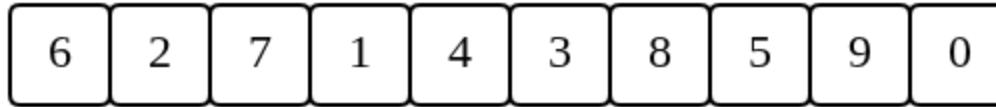


Bubble Sort - Implementation

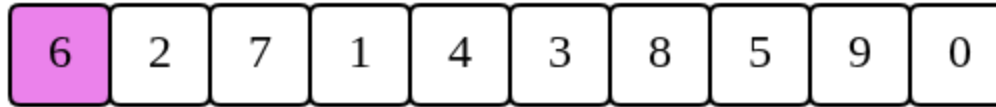
```
public static void sort(int[] array)
{
    sort(array, 0, array.length - 1);
}

private static void sort(int[] array, int low, int high)
{
    // outer loop for iterations
    for (int i = low; i < high; i++)
    {
        // inner loop for comparison and swapping
        for (int j = low; j < high - i; j++)
        {
            // swap if the current element is greater than the next element
            if (array[j] > array[j + 1])
                swap(array, j, j + 1);
        }
    }
}
```

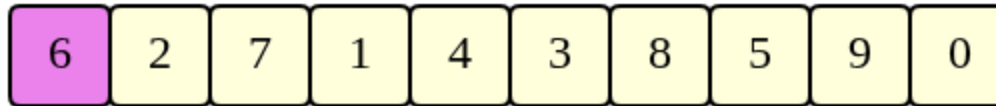
Selection Sort - Example



provide the unsorted array



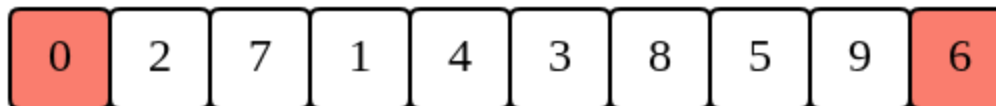
mark the current element



find the minimum value of the rest

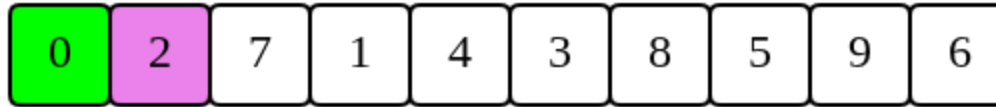


compare current element and min element

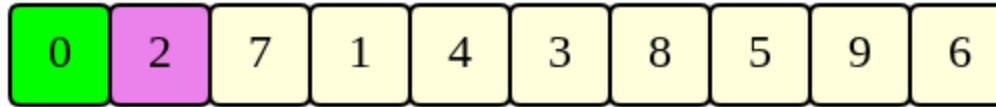


swap them because 0 should go before 6

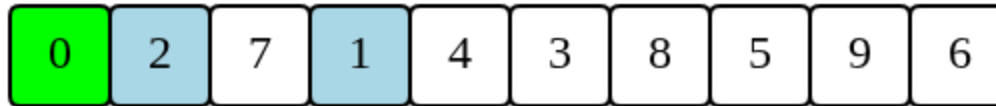
Selection Sort - Example (cont.)



mark new current element



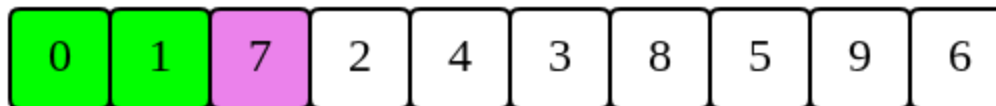
find the minimum value of the rest



compare current element and min element

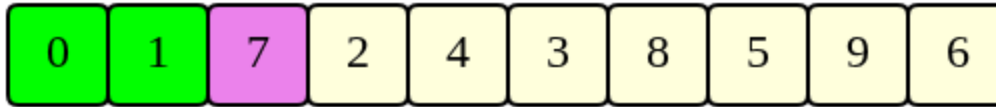


swap them because 1 should go before 2

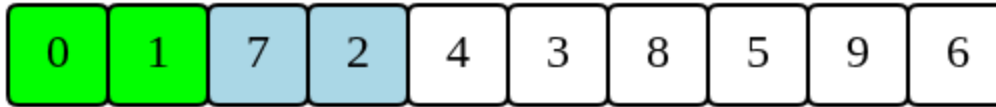


mark new current element

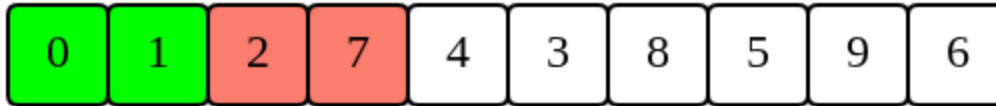
Selection Sort - Example (cont.)



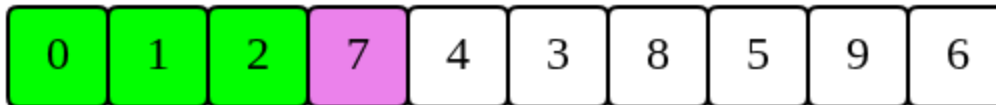
find the minimum value of the rest



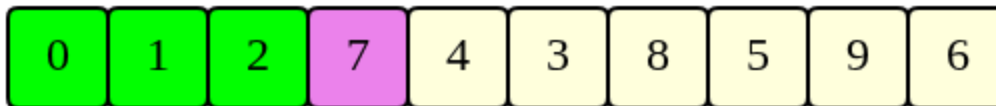
compare current element and min element



swap them because 2 should go before 7



mark new current element



find the minimum value of the rest

Selection Sort - Example (cont.)



compare current element and min element



swap them because 3 should go before 7



mark new current element



find the minimum value of the rest



compare current element and min element



Selection Sort - Example (cont.)



do nothing since 4 is less than 5



the array after 6th iteration



the array after 7th iteration



the array after 9th iteration



the array is now sorted



Selection Sort - Observation

- The main idea of algorithm is to **compare the current element** with **minimum value of the rest** to push the minimum element to the left of the array
- The implementation was to repeat **n** iterations where **n** is the number of elements in the array
- The implementation optimizes the algorithm of bubble sort to reduce the comparison of each element repeatedly in the array



Selection Sort - Implementation

```
public static void sort(int[] array)
{
    sort(array, 0, array.length - 1);
}

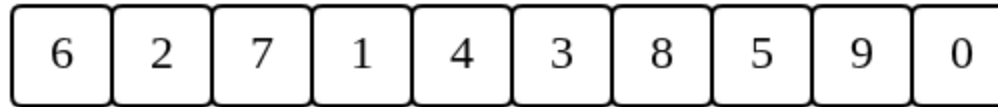
public static void sort(int[] array, int low, int high)
{
    int current, min;

    // outer loop for iterations
    for (int i = low; i < high; i++)
    {
        current = i; // make the current element
        min = i + 1; // find the minimum element in the unsorted range of array
        for (int j = min + 1; j < high + 1; j++)
        {
            if (array[min] > array[j])
                min = j; // update the min index
        }

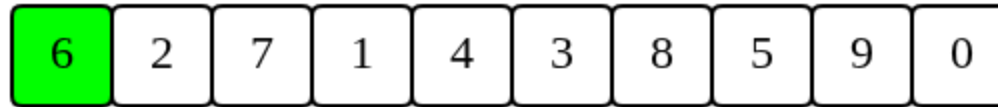
        // swap current element with the minimum element in unsorted range
        if (array[current] > array[min])
            swap(array, current, min);
    }
}
```

25

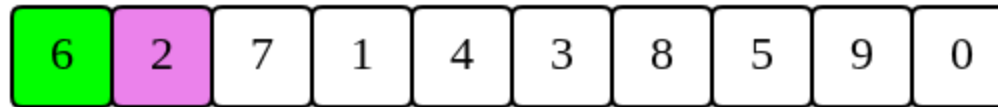
Insertion Sort - Example



provide the unsorted array



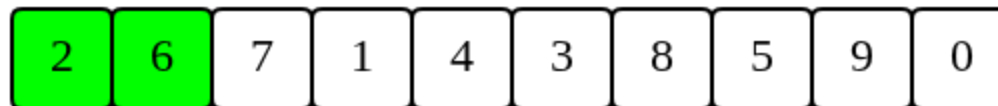
the first element is placed in sorted area



mark current element

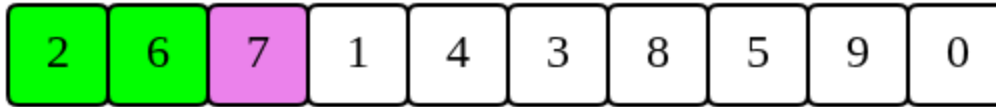


place current element in sorted area

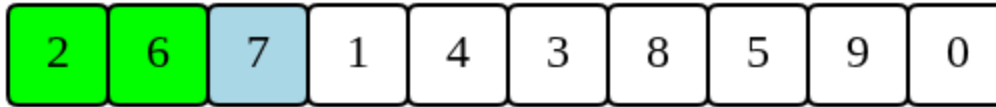


the array after 1st iteration

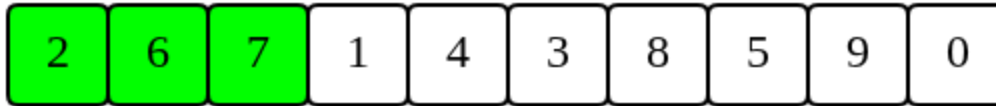
Insertion Sort - Example (cont.)



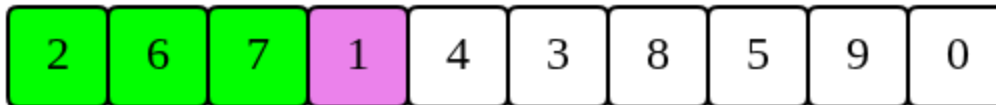
mark current element



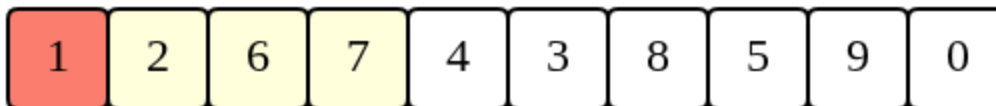
7 is greater than last element in sorted area



the array after 2nd iteration



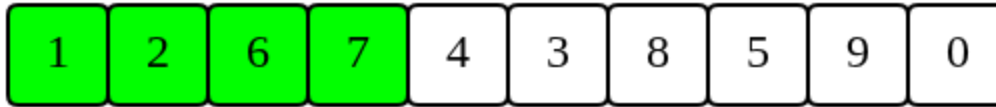
mark current element



place current element in sorted area



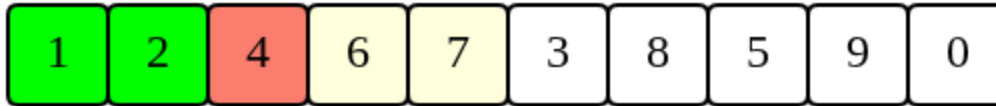
Insertion Sort - Example (cont.)



the array after 3rd iteration



mark current element



place current element in sorted area



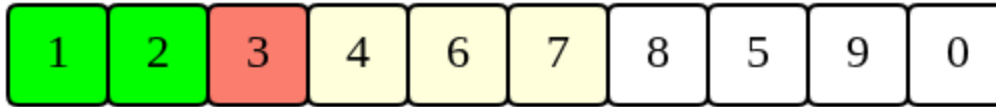
the array after 4th iteration



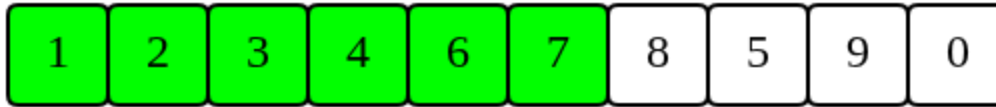
mark current element



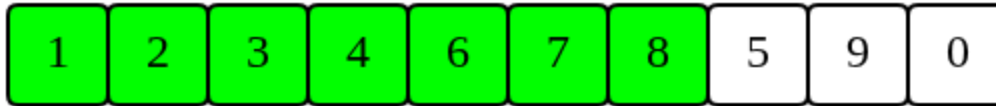
Insertion Sort - Example (cont.)



place current element in sorted area



the array after 5th iteration



the array after 6th iteration



the array after 9th iteration



the array is now sorted



Insertion Sort - Observation

- The main idea of the algorithm is to compare the current element with the sorted area (on the left side of the array) and place the current element in appropriate location to maintain the area sorted
- The implementation is maintain a sorted area for every single element in the array
- The current element compares with each element in the sorted area from the right (compare with element from larger to smaller) and stop comparing when it found the right place to put the current element in (optimization)



Insertion Sort - Implementation

```
public static void sort(int[] array)
{
    sort(array, 0, array.length - 1);
}

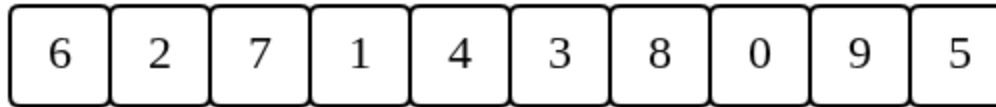
public static void sort(int[] array, int low, int high)
{
    int sorted, temp, j;

    for (int i = low + 1; i < high + 1; i++) // i is the index of the current element
    {
        sorted = low; // index of elements in the sorted portion of the array
        while (array[i] > array[sorted])
            sorted++; // find the correct position to insert the current element

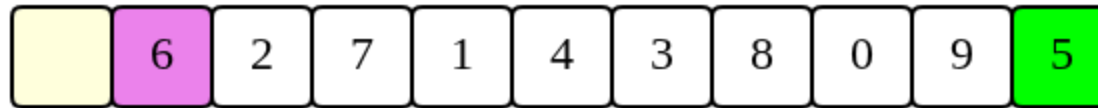
        // shift elements to the right to make space for insertion
        temp = array[i]; j = i;
        while (j > sorted)
        {
            array[j] = array[j - 1];
            j--;
        }
        array[sorted] = temp; // place current element to correct position
    }
}
```

31

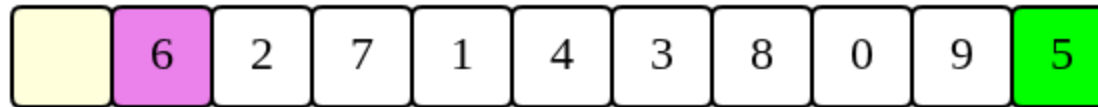
Quick Sort - Example



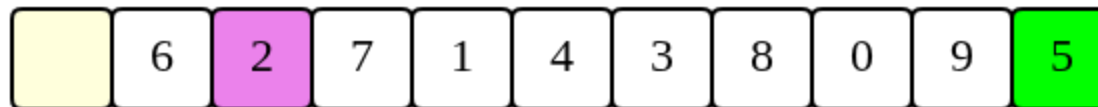
provide the unsorted array



initialize separator, current element, pivot



separator unchanged since current > pivot



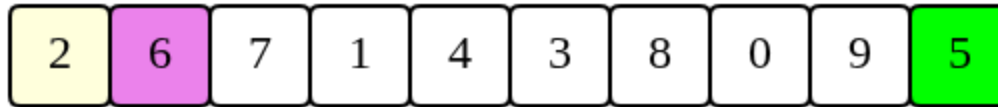
mark new current element



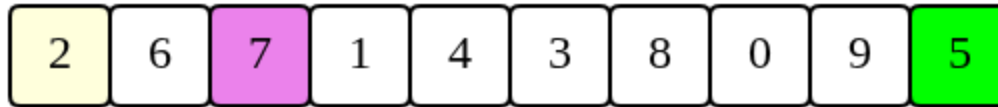
move separator since current < pivot ...



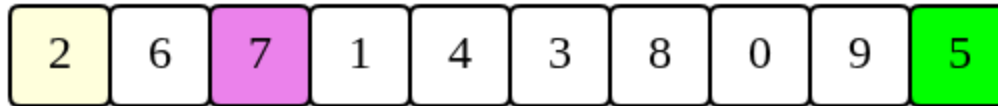
Quick Sort - Example (cont.)



... swap separator and current element



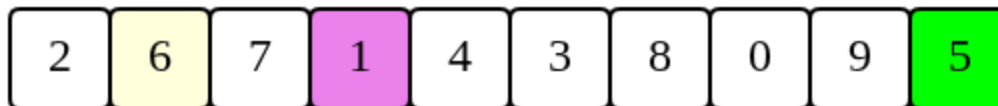
mark new current element



separator unchanged since current > pivot



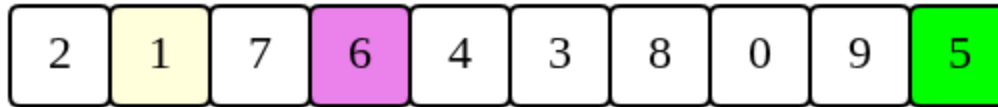
mark new current element



move separator since current < pivot ...



Quick Sort - Example (cont.)



... swap separator and current element



mark new current element



move separator since current < pivot ...



... swap separator and current element



mark new current element

Quick Sort - Example (cont.)



mark new current element



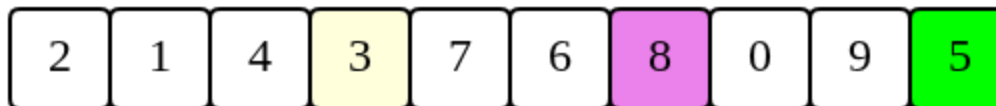
move separator since current < pivot ...



... swap separator and current element

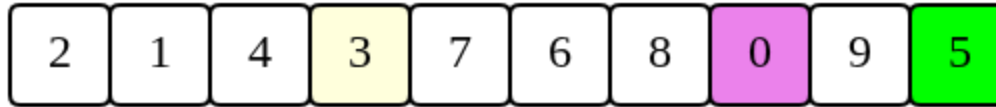


mark new current element



separator unchanged since current > pivot

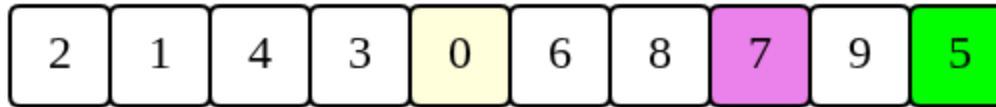
Quick Sort - Example (cont.)



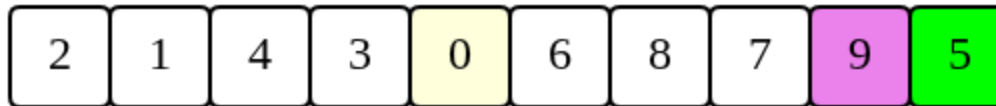
mark new current element



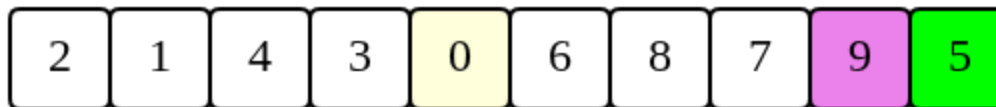
move separator since current < pivot ...



... swap separator and current element

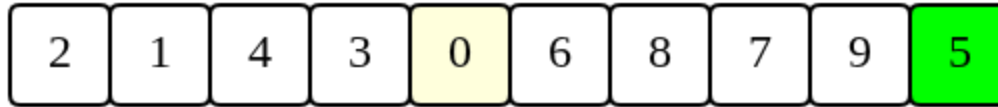


mark new current element

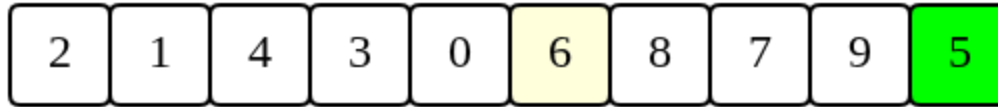


separator unchanged since current > pivot

Quick Sort - Example (cont.)



mark new current element (reached pivot)



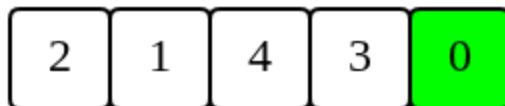
move separator ...



... swap separator and pivot

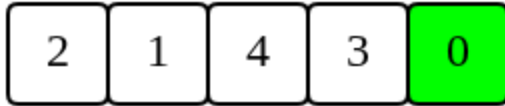


split in two partitions



pick a pivot for the first partition

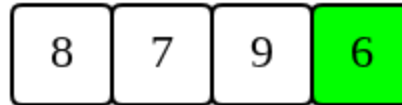
Quick Sort - Example (cont.)



apply algorithm recursively on partition ...



... until the partition is sorted



pick a pivot for the first partition



apply algorithm until partition is sorted



the array is now sorted



Quick Sort - Observation

- The main idea of the algorithm is to split the array into two partitions and recursively apply the algorithm on those two partitions
- The implementation is maintain two partitions that one partition has elements smaller than the pivot and the other has elements greater than the pivot
- The optimization idea is to choose a random pivot that can minimize the chance that you will encounter worst-case (quadratic time complexity) performance
- How to select the pivot:
 - Choosing first or last would cause worst-case performance for nearly-sorted or nearly-reverse-sorted data).
 - Choosing the middle element would also be acceptable in the majority of cases.



Quick Sort - Implementation

```
public static void sort(int[] array)
{
    sort(array, 0, array.length - 1);
}

private static void sort(
int[] array, int low, int high)
{
    if (low < high)
    {
        int sep = partition(array, low, high);

        // sort left subarray
        sort(array, low, sep - 1);
        // sort right subarray
        sort(array, sep + 1, high);
    }
}
```

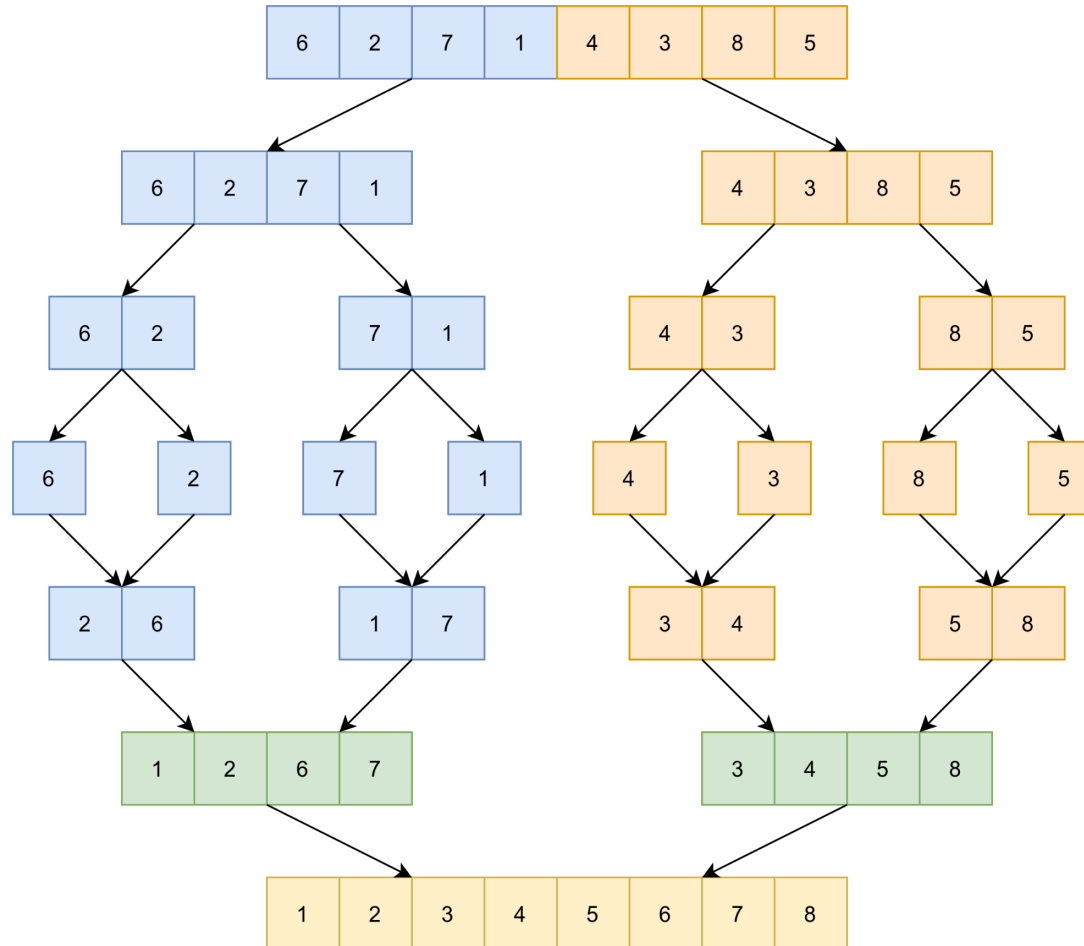
```
private static int partition(
int[] array, int low, int high)
{
    int pivot = high; // choose last element as pivot
    int separator = low - 1; // last index of smaller items
    int current = low;

    // run current until it reaches pivot
    while (current < pivot)
    {
        if (array[current] < array[pivot])
        {
            separator++; // move the separator
            swap(array, separator, current);
        }

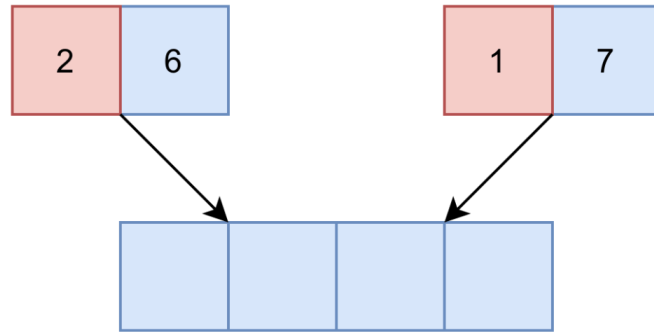
        current++; // mark new current element
    }

    swap(array, separator + 1, high);
    return separator + 1;
}
```

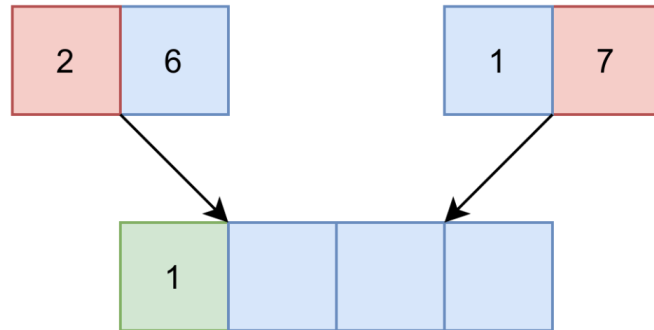

Merge Sort - Example



Merge Sort - Example (cont.)



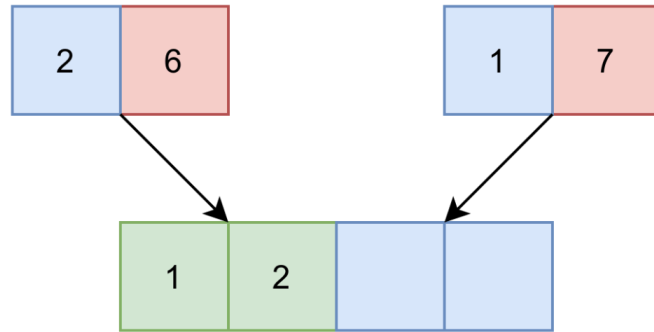
Compare each element in each part



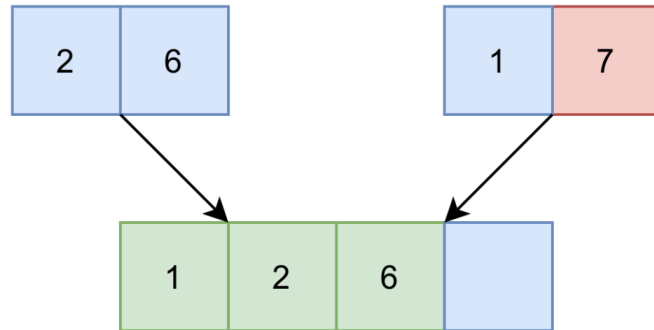
Put the smaller element into the new array
Then move to the next element to compare

1 is smaller than 2 so we put 1 into the new
array and move to element 7

Merge Sort - Example (cont.)

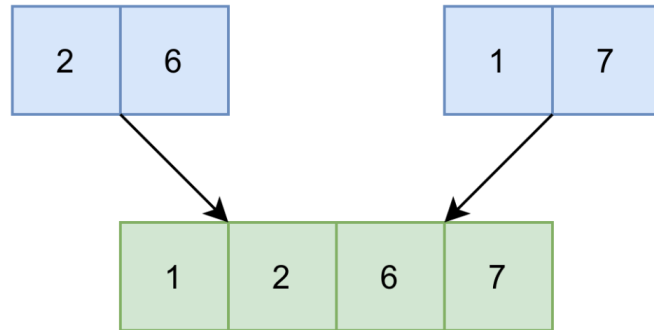


2 is smaller than 7 so we put 2 into the new array and move to element 6



6 is smaller than 7 so we put 6 into the new array and there is no element left

Merge Sort - Example (cont.)



Put all remaining elements left the other partition into the new array in order to finish the merge



We use this merged partition and again merge with another partition to new array





Merge Sort - Observation

- The main idea of the algorithm is to split the array into two partitions with the same portion (off by one if the length of the array is an odd number)
- The implementation is to rearrange the elements respectively from smaller to larger into a new array



Merge Sort - Implementation

```
public static void sort(int[] array)
{
    int[] temp = new int[array.length];
    sort(array, temp, 0, array.length - 1);
}

private static void sort(
int[] array, int[] temp, int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        // sort left half
        sort(array, temp, low, mid);

        // sort right half
        sort(array, temp, mid + 1, high);

        // merge sorted halves
        merge(array, temp, low, mid, high);
    }
}
```

```
private static void merge(
int[] array, int[] temp, int low, int mid, int high)
{
    // copy both halves into the temporary array
    for (int i = low; i <= high; i++)
        temp[i] = array[i];

    int i = low, j = mid + 1, k = low;

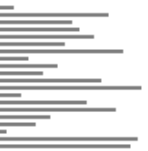
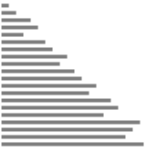
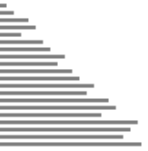
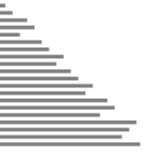
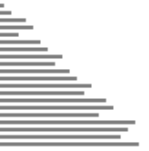
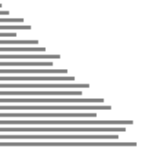
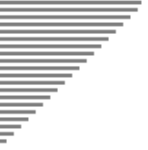
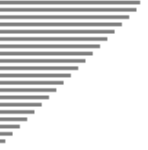
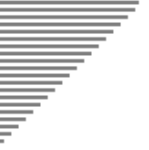
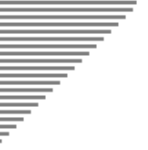
    // merge the temporary arrays back into array
    while (i <= mid && j <= high)
    {
        if (temp[i] <= temp[j])
            array[k] = temp[i++];
        else array[k] = temp[j++];
        k++;
    }

    // copy the remaining elements of left half
    while (i <= mid)
        array[k++] = temp[i++];
    // copy the remaining elements of right half
    while (j <= high)
        array[k++] = temp[j++];
}
```

Sorting Algorithm Complexity

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(\log(n))$
Mergesort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Timsort	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Heapsort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n)$	$O((n \log(n))^2)$	$O((n \log(n))^2)$	$O(1)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Sorting Algorithm Complexity (cont.)

	 Insertion	 Selection	 Bubble	 Shell	 Merge	 Heap	 Quick	 Quick3
 Random								
 Nearly Sorted								
 Reversed								
 Few Unique								

Questions & Answer





Thank You!

A black fountain pen with silver-colored accents and a silver nib is positioned diagonally below the end of the 'Thank You!' text. The pen is lying on its side, pointing towards the bottom right corner of the slide.